Complete Beginner's Guide

Multi-Agent Sales Forecasting System

A full, from-scratch explanation of what this project is, why each part exists, how the code works, and how to build it step by step — written for someone who is new to time-series forecasting and agentic systems.

1. What Exactly Is This Project?

Imagine a store that has been recording its daily sales for the past two or three years. The manager wants to know: how much will we sell next month? Normally, an analyst would clean the sales data, pick a forecasting technique, check whether anything looks unusual in the data, and write up a summary for management. This project builds four "agents" — specialized pieces of code, each with one job — that do this automatically, one after another:

- **Agent 1 (Data Preprocessing Agent):** Cleans the raw sales history and prepares it for forecasting.
- **Agent 2 (Forecasting Agent):** Predicts future sales using a forecasting model.
- **Agent 3 (Anomaly Detection Agent):** Flags any unusual spikes or drops in the data.
- **Agent 4 (Explanation Agent):** Writes a plain-English summary of the forecast and any anomalies found.

The rest of this document explains every concept needed and gives you working code for each agent.

2. Key Concepts Explained Simply

2.1 What is Time-Series Data?

Time-series data is simply data collected over time, in order — like daily sales, daily temperature, or monthly revenue. The key difference from normal ML data is that the **order matters**: yesterday's sales can help predict tomorrow's, so you cannot shuffle the rows randomly like you would with the loan dataset in a normal ML project.

2.2 Trend, Seasonality, and Noise

- **Trend:** The general long-term direction — are sales slowly going up or down over months/years?
- **Seasonality:** Repeating patterns at fixed intervals — e.g., sales are always higher in December, or every Saturday.
- **Noise:** Random day-to-day fluctuation that no model can perfectly predict.

> A good forecasting model separates these three things internally and re-combines them to make its prediction. The library used in this project (Facebook Prophet) does this automatically for you.

2.3 What is Facebook Prophet?

Prophet is a forecasting library made by Meta (Facebook), specifically designed to be beginner-friendly. Unlike classical time-series methods (like ARIMA) which require you to manually tune several statistical parameters, Prophet automatically detects trend and seasonality with almost no configuration. You just give it a table with two columns — a date column (named 'ds') and a value column (named 'y') — and call model.fit() and model.predict(), similar to scikit-learn.

2.4 What Counts as an "Anomaly" Here?

An anomaly is simply a data point that is much higher or lower than what is "normal" for that period. This project uses a simple, well-known statistical rule: calculate the average and standard deviation of recent sales, and flag any value that is more than 2 or 3 standard deviations away from that average. This requires no machine learning at all — just arithmetic — which keeps this part of the project light.

2.5 What Does the Explanation Agent Do?

This agent does not forecast or detect anomalies itself — it takes the numeric results from Agents 2 and 3 and asks a Large Language Model (like GPT or Claude, via their API) to turn those numbers into a short, readable business report. This is exactly what you would do by typing numbers into ChatGPT/Claude and asking it to summarize them — except it happens automatically inside your Python code.

3. Environment Setup

Do this once, at the very start. Open a terminal (Command Prompt / VS Code terminal) and run:

```
# 1. Create a project folder and move into it
mkdir agentic_sales_forecast
cd agentic_sales_forecast

# 2. (Optional but recommended) Create a virtual environment
python -m venv venv
venv\Scripts\activate        # Windows
# source venv/bin/activate    # Mac/Linux

# 3. Install all required libraries
pip install pandas prophet scikit-learn matplotlib openai
```

Then download a beginner-friendly dataset, for example the **Superstore Sales Dataset** or the **Store Sales - Time Series Forecasting** dataset from Kaggle, and save it as **sales_data.csv** inside this same folder. Make sure it has at least a date column and a sales/quantity column.

> Note: installing Prophet can occasionally need extra build tools on Windows. If pip install prophet fails, try: pip install pystan==2.19.1.1 first, then pip install prophet again, or search "install fbprophet windows" for the latest guidance.

4. Agent 1 — Data Preprocessing Agent

Raw sales data is often messy: some days might be missing entirely (e.g., store closed), dates might be in inconsistent formats, and the data might need to be aggregated (e.g., many transactions per day need to be summed into one daily total). This agent handles all of that and creates a few extra helper columns used later.

File: data_preprocessing_agent.py

```
import pandas as pd


class DataPreprocessingAgent:
    """
    Agent responsible for cleaning raw sales data and preparing it
    for time-series forecasting.
    """

    def preprocess(self, df: pd.DataFrame, date_column: str, sales_column: str) -> pd.DataFrame:
        df = df.copy()

        # 1. Convert date column to proper datetime format
        df[date_column] = pd.to_datetime(df[date_column])

        # 2. If there are multiple rows per day (e.g., multiple transactions),
        #    aggregate them into one total per day
        df = df.groupby(date_column, as_index=False)[sales_column].sum()

        # 3. Sort by date -- very important for time series!
        df = df.sort_values(date_column)

        # 4. Fill in any missing dates (e.g., days with zero recorded sales)
        full_date_range = pd.date_range(
            start=df[date_column].min(), end=df[date_column].max(), freq="D"
        )
        df = df.set_index(date_column).reindex(full_date_range)
        df[sales_column] = df[sales_column].fillna(0)
        df.index.name = date_column
        df = df.reset_index()

        # 5. Create simple helper features (useful for anomaly detection later)
        df["day_of_week"] = df[date_column].dt.day_name()
        df["month"] = df[date_column].dt.month

        # 6. Create a lag feature: sales from 7 days ago (helps some ML models)
        df["sales_lag_7"] = df[sales_column].shift(7)

        print(f"Preprocessing complete. Final shape: {df.shape}")
        return df


# Quick standalone test
if __name__ == "__main__":
    raw_df = pd.read_csv("sales_data.csv")
    agent = DataPreprocessingAgent()
    clean_df = agent.preprocess(raw_df, date_column="Order_Date", sales_column="Sales")
    print(clean_df.head(10))
```

What to notice: the most important line is df.sort_values(date_column) — time-series data must always be in date order. The reindex() step fills in any missing calendar days with zero sales, so the forecasting model doesn't get confused by gaps.

5. Agent 2 — Forecasting Agent

This agent takes the cleaned daily sales data and predicts sales for the next N days using Facebook Prophet. Prophet expects a DataFrame with exactly two columns named 'ds' (date) and 'y' (the value to predict) — so we rename our columns to match before training.

File: forecasting_agent.py

```
import pandas as pd
from prophet import Prophet
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np


class ForecastingAgent:
    """
    Agent responsible for training a forecasting model and predicting
    future sales values.
    """

    def __init__(self):
        self.model = Prophet(yearly_seasonality=True, weekly_seasonality=True)

    def forecast(self, df: pd.DataFrame, date_column: str, sales_column: str,
                 periods: int = 30, test_days: int = 30):
        # Prophet requires columns named exactly 'ds' and 'y'
        prophet_df = df[[date_column, sales_column]].rename(
            columns={date_column: "ds", sales_column: "y"}
        )

        # Hold out the last `test_days` for evaluation
        train_df = prophet_df.iloc[:-test_days]
        test_df = prophet_df.iloc[-test_days:]

        # Step 1: Fit / Train
        self.model.fit(train_df)

        # Step 2: Predict for the test period + future periods
        future = self.model.make_future_dataframe(periods=test_days + periods)
        forecast_df = self.model.predict(future)

        # Step 3: Evaluate on the held-out test period
        predicted_test = forecast_df.set_index("ds").loc[test_df["ds"], "yhat"].values
        actual_test = test_df["y"].values

        mae = mean_absolute_error(actual_test, predicted_test)
        rmse = np.sqrt(mean_squared_error(actual_test, predicted_test))

        metrics = {"MAE": round(mae, 2), "RMSE": round(rmse, 2)}
        print(f"Forecast evaluation -> MAE: {metrics['MAE']}, RMSE: {metrics['RMSE']}")

        # Return only the future predictions (beyond the original data)
        future_only = forecast_df.tail(periods)[["ds", "yhat", "yhat_lower", "yhat_upper"]]
        return future_only, metrics, forecast_df


# Quick standalone test (run after data_preprocessing_agent.py works)
```

```
if __name__ == "__main__":
    from data_preprocessing_agent import DataPreprocessingAgent

    raw_df = pd.read_csv("sales_data.csv")
    prep_agent = DataPreprocessingAgent()
    clean_df = prep_agent.preprocess(raw_df, date_column="Order_Date", sales_column="Sales")

    forecast_agent = ForecastingAgent()
    future_only, metrics, full_forecast = forecast_agent.forecast(
        clean_df, date_column="Order_Date", sales_column="Sales", periods=30
    )
    print(future_only)
```

What to notice: yhat is Prophet's predicted value; yhat_lower and yhat_upper are the lower and upper bounds of its confidence interval (i.e., "it probably falls somewhere in this range"). MAE tells you, on average, how many units the forecast was off by — smaller is better.

6. Agent 3 — Anomaly Detection Agent

This agent looks at the sales history (and optionally the forecast) and flags days where sales were unusually high or low, using a simple and well-known statistical technique called the rolling mean and standard deviation method — no machine learning training required.

File: anomaly_detection_agent.py

```
import pandas as pd


class AnomalyDetectionAgent:
    """
    Agent responsible for flagging unusually high or low sales values,
    using a rolling mean + standard deviation threshold.
    """

    def __init__(self, window: int = 30, threshold: float = 2.5):
        self.window = window        # how many past days to consider "normal"
        self.threshold = threshold  # how many standard deviations counts as an anomaly

    def detect(self, df: pd.DataFrame, date_column: str, sales_column: str) -> pd.DataFrame:
        df = df.copy()

        # Rolling average and rolling standard deviation over the chosen window
        df["rolling_mean"] = df[sales_column].rolling(self.window, min_periods=5).mean()
        df["rolling_std"] = df[sales_column].rolling(self.window, min_periods=5).std()

        # Calculate how many standard deviations away each point is
        df["z_score"] = (df[sales_column] - df["rolling_mean"]) / df["rolling_std"]

        df["is_anomaly"] = df["z_score"].abs() > self.threshold
        df["anomaly_type"] = df["z_score"].apply(
            lambda z: "spike" if z > self.threshold else ("drop" if z < -self.threshold else "normal")
        )

        anomalies = df[df["is_anomaly"]][[date_column, sales_column, "anomaly_type"]]
        print(f"Found {len(anomalies)} anomalies out of {len(df)} data points.")
        return anomalies


# Quick standalone test
if __name__ == "__main__":
    import pandas as pd
    from data_preprocessing_agent import DataPreprocessingAgent

    raw_df = pd.read_csv("sales_data.csv")
    prep_agent = DataPreprocessingAgent()
    clean_df = prep_agent.preprocess(raw_df, date_column="Order_Date", sales_column="Sales")

    anomaly_agent = AnomalyDetectionAgent(window=30, threshold=2.5)
    anomalies = anomaly_agent.detect(clean_df, date_column="Order_Date", sales_column="Sales")
    print(anomalies)
```

What to notice: a "z-score" simply tells you how many standard deviations a value is away from the average. A z-score of 3 means "this is a very unusual value compared to recent history." You can make the detector stricter (fewer anomalies flagged) by raising the threshold, or looser by lowering it.

7. Agent 4 — Explanation Agent

This agent converts the forecast and anomaly results into a plain-English report. As with the earlier project, we show a simple no-API version first, then an LLM-powered version.

7.1 Version A — Template-Based (no API key needed)

```
class ExplanationAgentSimple:
    """
    Generates a plain-English sales forecast summary using a fixed template.
    """

    def explain(self, future_only, metrics: dict, anomalies) -> str:
        avg_forecast = future_only["yhat"].mean()
        lines = [
            f"Based on historical sales data, the average predicted sales for the upcoming period "
            f"is approximately {avg_forecast:.0f} units per day.",
            f"The forecasting model achieved a Mean Absolute Error (MAE) of {metrics['MAE']} and an "
            f"RMSE of {metrics['RMSE']} on recent historical data, indicating how far off predictions
            f"typically are.",
        ]
        if len(anomalies) > 0:
            lines.append(
                f"{len(anomalies)} unusual date(s) were detected in the historical data, which may be
                f"worth reviewing for data errors or real business events:"
            )
            for _, row in anomalies.iterrows():
                lines.append(f"  - {row[0].date()}: {row['anomaly_type']} in sales")
        else:
            lines.append("No unusual spikes or drops were detected in the historical data.")

        return "\n".join(lines)
```

7.2 Version B — Using an LLM API

```
from openai import OpenAI

class ExplanationAgentLLM:
    """
    Generates a natural-language sales forecast report by asking an LLM
    to interpret the forecast and anomaly results.
    """

    def __init__(self):
        self.client = OpenAI()   # reads OPENAI_API_KEY automatically from your environment

    def explain(self, future_only, metrics: dict, anomalies) -> str:
        forecast_summary = future_only[["ds", "yhat"]].to_string(index=False)
        anomaly_summary = anomalies.to_string(index=False) if len(anomalies) > 0 else "None detected."

        prompt = f"""
You are writing a short sales forecast report for a store manager who is not technical.

Forecast for the upcoming period:
{forecast_summary}

Model evaluation metrics: {metrics}
```

```
Anomalies detected in historical data:
{anomaly_summary}

Write a short, clear, 4-5 sentence business summary explaining the forecast trend,
how reliable it is, and whether the manager should look into any flagged anomalies.
Avoid technical jargon.
"""
        response = self.client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": prompt}]
        )
        return response.choices[0].message.content
```

To set your API key on Windows: setx OPENAI_API_KEY "your-key-here" (then restart the terminal). On Mac/Linux: export OPENAI_API_KEY="your-key-here". Never write the key directly inside your code file.

8. Putting It All Together — main.py

This is the file you actually run. It calls all four agents one after another and prints the final report.

```
import pandas as pd
from data_preprocessing_agent import DataPreprocessingAgent
from forecasting_agent import ForecastingAgent
from anomaly_detection_agent import AnomalyDetectionAgent
from explanation_agent import ExplanationAgentSimple  # swap for ExplanationAgentLLM later

DATE_COLUMN = "Order_Date"
SALES_COLUMN = "Sales"

def main():
    print("STEP 1: Loading raw sales data...")
    raw_df = pd.read_csv("sales_data.csv")

    print("\nSTEP 2: Preprocessing data...")
    prep_agent = DataPreprocessingAgent()
    clean_df = prep_agent.preprocess(raw_df, date_column=DATE_COLUMN, sales_column=SALES_COLUMN)

    print("\nSTEP 3: Forecasting future sales...")
    forecast_agent = ForecastingAgent()
    future_only, metrics, full_forecast = forecast_agent.forecast(
        clean_df, date_column=DATE_COLUMN, sales_column=SALES_COLUMN, periods=30
    )

    print("\nSTEP 4: Detecting anomalies...")
    anomaly_agent = AnomalyDetectionAgent(window=30, threshold=2.5)
    anomalies = anomaly_agent.detect(clean_df, date_column=DATE_COLUMN, sales_column=SALES_COLUMN)

    print("\nSTEP 5: Generating explanation...")
    explanation_agent = ExplanationAgentSimple()
    explanation = explanation_agent.explain(future_only, metrics, anomalies)

    print("\n================ FINAL REPORT ================")
    print(explanation)
    print("==============================================")

if __name__ == "__main__":
    main()
```

Run it with: `python main.py` — make sure all four agent files are in the same folder as main.py and sales_data.csv.

9. What You Should See When You Run It

```
STEP 1: Loading raw sales data...

STEP 2: Preprocessing data...
Preprocessing complete. Final shape: (1035, 6)

STEP 3: Forecasting future sales...
Forecast evaluation -> MAE: 182.45, RMSE: 231.60

STEP 4: Detecting anomalies...
Found 6 anomalies out of 1035 data points.

STEP 5: Generating explanation...

================ FINAL REPORT ================
Based on historical sales data, the average predicted sales for the
upcoming period is approximately 1420 units per day.
The forecasting model achieved a Mean Absolute Error (MAE) of 182.45
and an RMSE of 231.6 on recent historical data, indicating how far
off predictions typically are.
6 unusual date(s) were detected in the historical data, which may be
worth reviewing for data errors or real business events:
  - 2023-12-24: spike in sales
  - 2023-12-25: drop in sales
  - 2024-01-01: drop in sales
  ...
==============================================
```

10. Common Errors and How to Fix Them

Error Message	Likely Cause	Fix
ImportError: cannot import name 'Prophet'	Prophet not installed correctly	Run: pip install prophet (may need pystan first)
ValueError: Dataframe has less than 2 non-NaN rows	Too little data, or all values are missing	Check your CSV has enough rows of real sales data
KeyError: 'Order_Date'	Column name typed wrong / doesn't match your CSV	Open the CSV and check the exact column name/spelling
FileNotFoundError: sales_data.csv	CSV not in the same folder as your script	Move the CSV next to your .py files, or use full path

11. Glossary — Words You'll Hear a Lot

Time series: Data collected in order over time, where the sequence/order matters.

Trend: The long-term upward or downward direction of the data.

Seasonality: A pattern that repeats at regular intervals, like every week or every December.

Forecast: A prediction of future values based on past data.

MAE: Mean Absolute Error — average size of the forecast's mistakes, in the original units (e.g., units sold).

RMSE: Root Mean Squared Error — similar to MAE but penalizes large mistakes more heavily.

Rolling window: A moving "lookback" period (e.g., the last 30 days) used to calculate a local average.

Z-score: How many standard deviations a value is from the average; used to flag anomalies.

Anomaly: A data point that is unusually high or low compared to recent normal behavior.

LLM: Large Language Model, e.g. GPT or Claude, used here only to write the explanation text.

12. Tips for the Viva / Demo

- Be ready to explain what each of the four agents does in your own words — you don't need to memorize the code, just the purpose of each stage.
- If asked "why Prophet and not a plain regression model", the answer is: Prophet is specifically built to automatically handle trend and seasonality in time-series data, which a plain regression model does not do out of the box.
- If asked why anomaly detection doesn't use ML, explain that a simple statistical rule (rolling mean + standard deviation) is fast, easy to explain, and works well for this scale of project — more complex ML-based anomaly detection is a good final-year extension.
- Have the sample output (Section 9) and a forecast chart ready to show on screen during the demo, even if the live run is slow.